Jood Alsaedi	446009064	20%
Hanin Alharthi	446005953	20%
Jana Saati	446017177	20%
Elan Alghamdi	446001526	20%
Leen Alasiri	446005664	20%

Lab 1: Data Loading & Inspection

```
import pandas as pd

from google.colab import files
uploaded = files.upload()

# تحميل البيانات
df = pd.read_csv('sleep_study_1000.csv')
```

```
اختيار ملفات sleep_study_1000.csv
sleep_study_1000.csv(text/csv) - 105648 bytes, last modified: 100% - ٢٠٢٦٢/٨/٥ done
Savingsleep_study_1000.csv  to sleep_study_1000.csv
```

```
# حجم البيانات (عدد الصفوف والأعمدة)
print("Dataset Shape (Rows, Columns):", df.shape)
```

```
Dataset Shape (Rows, Columns): (1000, 15)
```

```
# عرض أول ٥ صفوف
print(df.head())
```

```
   ID Age Gender                Bedtime                WakeupTime \
0   1      80 Female 2025-09-30 07:32:17 2025-09-30  13:49:16.669051
1   2   24     Male 2025-06-29 20:59:50 2025-06-30  04:09:10.207162
2   3   37     Male 2025-12-24 21:28:32 2025-12-25 03:31:34.257053
3   4     68 Female 2025-02-22 00:25:33 2025-02-22 07:26:37.046703
4   5   58     Male 2025-09-02 12:31:53 2025-09-02 21:17:46.256228

    SleepDuration   SleepEfficiency REMSleepPercentage  DeepSleepPercentage \
0        6.283241             0.57                  15                   35
1        7.155613             0.91                  29                   68
2        6.050627             0.58                  15                   35
3        7.017791             0.88                  28                   44
4        8.764793             0.95                  28                   40

     LightSleepPercentage Awakenings  CaffeineConsumption  AlcoholConsumption \
0                      50          0                 25.0                   1
1                       3          4                 50.0                   0
2                      50          3                 50.0                   0
3                      28          1                 50.0                   0
4                      32          4                 25.0                   4

   SmokingStatus  ExerciseFrequency
0            Yes                  1
1             No                  2
2             No                  5
3            Yes                  4
4             No                  4
```

```
# فحص البيانات
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype -
--- ------                --------------  ----
 0   ID                    1000  non-null  int64
 1   Age                   1000  non-null  int64
 2   Gender                1000  non-null  object
 3   Bedtime               1000 non-null   object
 4   WakeupTime            1000 non-null   float64
 5   SleepDuration         1000 non-null   float64
 6   SleepEfficiency       1000  non-null  int64
 7   REMSleepPercentage    1000  non-null  int64
 8   DeepSleepPercentage   1000 non-null   int64
 9   LightSleepPercentage  1000 non-null   int64
 10 Awakenings             744 non-null    float64
 11 CaffeineConsumption    1000 non-null   int64
 12 AlcoholConsumption     1000  non-null  object
 13 SmokingStatus          1000 non-null   int64
 14 ExerciseFrequency
dtypes: float64(3), int64(8), object(4)
memory usage: 117.3+ KB
```

```
# الإحصاءات الوصفية
print(df.describe())
```

```
                ID              Age   SleepDuration  SleepEfficiency     \
count      1000.000000  1000.000000  1000.000000      1000.000000
mean        500.500000    50.125000     7.027443         0.720150
std         288.819436    18.076397     1.135365         0.128139
min           1.000000    18.000000     5.002335         0.500000
25%         250.750000    18.000000     6.076980         0.610000
50%         500.500000    51.000000     6.981505         0.720000
75%         750.250000    65.000000     8.039587         0.830000
max        1000.000000    80.000000     8.998650         0.950000


       REMSleepPercentage  DeepSleepPercentage  LightSleepPercentage   \
count         1000.000000          1000.000000           1000.000000
mean            22.460000            47.357000             30.183000
std              4.589059            13.097293             13.788968
min             15.000000            25.000000              1.000000
25%             18.000000            36.000000             18.000000
50%             22.000000            47.000000             30.000000
75%             27.000000            59.000000             42.000000
max             30.000000            70.000000             59.000000
          Awakenings  CaffeineConsumption  AlcoholConsumption  ExerciseFrequency
count  1000.000000           744.000000          1000.000000        1000.000000
mean       2.539000            25.168011             2.560000           2.565000
std        1.726691            20.446036             1.746267           1.712515
min        0.000000             0.000000             0.000000           0.000000
25%        1.000000             0.000000             1.000000           1.000000
50%        3.000000            25.000000             3.000000           3.000000
75%        4.000000            50.000000             4.000000           4.000000
max        5.000000            50.000000             5.000000           5.000000
```

```
# عرض أسماء الأعمدة
print(df.columns)
```

```
Index(['ID', 'Age', 'Gender', 'Bedtime', 'WakeupTime', 'SleepDuration',
       'SleepEfficiency', 'REMSleepPercentage', 'DeepSleepPercentage',
       'LightSleepPercentage', 'Awakenings', 'CaffeineConsumption',
       'AlcoholConsumption', 'SmokingStatus', 'ExerciseFrequency'],
      dtype='object')
```

```
# إعادة تسمية الأعمدة لجعلها أكثر وضوحاً
df.rename(columns={
    'ID': 'Participant_ID',
    'SleepDuration': 'Sleep_Duration_Hours',
    'SleepEfficiency': 'Sleep_Efficiency_Percentage',
    'CaffeineConsumption': 'Caffeine_Intake_mg'
}, inplace=True)

#عرض الأسماءالجديدة
print(df.head())
```

```
   Participant_ID Age Gender                Bedtime  \
0               1      80 Female 2025-09-30 07:32:17
1               2   24     Male 2025-06-29 20:59:50
2               3   37     Male 2025-12-24 21:28:32
3               4     68 Female 2025-02-22 00:25:33
4               5   58     Male 2025-09-02 12:31:53

                WakeupTime  Sleep_Duration_Hours  \
```

```
0  2025-09-30   13:49:16.669051            6.283241
1  2025-06-30   04:09:10.207162            7.155613
2  2025-12-25 03:31:34.257053              6.050627
3  2025-02-22 07:26:37.046703              7.017791
4  2025-09-02 21:17:46.256228              8.764793

      Sleep_Efficiency_Percentage REMSleepPercentage  DeepSleepPercentage \
0                            0.57                 15                   35
1                            0.91                 29                   68
2                            0.58                 15                   35
3                            0.88                 28                   44
4                            0.95                 28                   40

      LightSleepPercentage Awakenings  Caffeine_Intake_mg  AlcoholConsumption \
0                       50          0                25.0                   1
1                        3          4                50.0                   0
2                       50          3                50.0                   0
3                       28          1                50.0                   0
4                       32          4                25.0                   4

   SmokingStatus  ExerciseFrequency
0            Yes                  1
1             No                  2
2             No                  5
3            Yes                  4
4             No                  4
```

1. What does each function show?

- df.head(): Displays the first five rows of the dataset to preview the data structure and inspect the initial records.
- df.shape: Shows the dataset dimensions, which are 1,000 rows and 15 columns.
- df.columns: Lists all column names in the dataset.
- df.info(): Provides a summary of data types and non-null values, helping identify missing data.
- df.describe(): Generates statistical summaries for numerical columns, including the mean, standard deviation, minimum, maximum, and quartiles.

2. Initial Observations can be made about the dataset

- Structure: The dataset contains 1,000 records and 15 features related to demographics, habits, and sleep measurements.
- Missing Data: The CaffeineConsumption column contains only 744 non-null values, indicating 256 missing entries that require preprocessing or imputation.
- Key Metrics:
  - Participants' ages range from 18 to 80 years, with an average age of approximately 50 years.
  - Participants wake up an average of 2.5 times per night.
  - The average REM sleep percentage is approximately 22.46%.

At the end of the inspection phase, several columns were renamed to maintain a cleaner and more consistent naming convention across the dataset.

Lab 2: Data Quality Assessment

```
import random
import matplotlib.pyplot as plt
import seaborn as sns

التحقق من القيم المفقودة قبل البدء #
print("Checking for missing values:")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

```
Checking for missing values:
Caffeine_Intake_mg    256
dtype: int64
```

```
فحص وعرض عدد الصفوف المكررة قبل التنظيف#
print("Number of duplicate rows before cleaning:", df.duplicated().sum())

# drop_duplicates() لايوجد صفوف مكررة في البيانات إذن لايلزم تنظيفها باستعمال
```

```
Number of duplicate rows before cleaning: 0
```

```
فحص التسميات للتأكد من تناسق النصوص #
print(df['Gender'].value_counts())
```

```
Gender
Male      513
Female    487
Name: count, dtype: int64
```

```
فحص القيم الصغرى و العظمى للتأكد من عدم وجود قيم سالبة أو مستحيلة #
print(df.describe().loc[['min', 'max']])
```

```
      Participant_ID   Age Sleep_Duration_Hours Sleep_Efficiency_Percentage   \
min              1.0  18.0              5.002335                         0.50
max           1000.0  80.0              8.998650                         0.95

       REMSleepPercentage DeepSleepPercentage  LightSleepPercentage \
min                  15.0                 25.0                   1.0
max                  30.0                 70.0                  59.0

      Awakenings  Caffeine_Intake_mg  AlcoholConsumption ExerciseFrequency
min          0.0                 0.0                 0.0               0.0
max          5.0                50.0                 5.0               5.0
```

```
التحقق من أنواع البيانات إذا كانت مختلطة أو خاطئة #
print(df.dtypes)
```

```
Participant_ID                int64
Age                           int64
Gender                       object
Bedtime                      object
WakeupTime                   object
Sleep_Duration_Hours        float64
Sleep_Efficiency_Percentage float64
REMSleepPercentage            int64
DeepSleepPercentage           int64
LightSleepPercentage          int64
Awakenings                    int64
Caffeine_Intake_mg          float64
AlcoholConsumption            int64
SmokingStatus                object
ExerciseFrequency             int64
dtype: object
```

1. List of All Detected Issues

During the initial inspection of the dataset, exactly two main data quality issues were identified:

1. **Missing Values** in the `Caffeine_Intake_mg` column.
2. **Incorrect Data Types** in the `Bedtime` and `WakeupTime` columns.

Note: No duplicate records, inconsistent categorical labels, or invalid/impossible numerical ranges were found.

2. How Each Issue Was Found & Evidence

Issue 1: Missing Values

- **How it was found:** We applied the `.isnull().sum()` function to check for any empty or null entries across the dataset.
- **Evidence:** The column `Caffeine_Intake_mg` contains **256 missing values**, while all other features are 100% complete (1,000 non-null values).

- **Why it is a problem:** Missing data can reduce statistical power, distort descriptive percentages, and cause errors or bias when training machine learning models if not properly handled.

Issue 2: Incorrect Data Types

- **How it was found:** We inspected the structural metadata and format of the columns using the `.dtypes` attribute.
- **Evidence:** Both Bedtime and WakeupTime columns are stored as **object (text/strings)** instead of the proper datetime format.
- **Why it is a problem:** Text formats treat dates and times as literal characters. This prevents us from performing mathematical chronological operations, such as calculating the exact sleep duration or grouping data by hours.

Lab 3: Data Cleaning & Exploratory Data Analysis (EDA)

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# 1. Handle Missing Values

# الطريقة الاولى: تعبئة القيم المفقودة لعمود الكافيين بالوسيط
caffeine_median = df['Caffeine_Intake_mg'].median()
df['Caffeine_Intake_mg'] = df['Caffeine_Intake_mg'].fillna(caffeine_median)

print("Missing values in 'Caffeine_Intake_mg' after median filling:", df['Caffeine_Intake_mg'].isnull().sum())
```

```
Missing values in 'Caffeine_Intake_mg' after median filling: 0
```

```
# الطريقة الثانية: اجراء الحذف كخطوة نهائية لضمان خلو البيانات من أي قيم مفقودة
df.dropna(inplace=True)


print("\nTotal missing values after handling (dropna):", df.isnull().sum().sum())
print("Dataset shape after dropna:", df.shape)
```

```
Total missing values after handling (dropna): 0
Dataset shape after dropna: (1000, 15)
```

```
# 2. Remove Duplicates

# (سيعمل كخطوة احترازية وان يحذف أي شيء لأن النتيجة صفر)
df.drop_duplicates(inplace=True)
print("Number of duplicate rows after cleaning:", df.duplicated().sum())
```

```
Number of duplicate rows after cleaning: 0
```

```
# 3. Fix Data Types

# تحويل أعمدة الأوقات النصية إلى صيغة وقت حقيقية
df['Bedtime'] = pd.to_datetime(df['Bedtime'])
df['WakeupTime'] = pd.to_datetime(df['WakeupTime'])

# طباعة الأنواع الجديدة للتأكد
print("")
print("New Data Types After Conversion")
print(df.dtypes)
```

```
New Data Types After Conversion
Participant_ID                      int64
Age                                 int64
Gender                             object
Bedtime                    datetime64[ns]
WakeupTime                 datetime64[ns]
Sleep_Duration_Hours              float64
Sleep_Efficiency_Percentage       float64
REMSleepPercentage                  int64
DeepSleepPercentage                 int64
LightSleepPercentage                int64
Awakenings                          int64
Caffeine_Intake_mg                float64
AlcoholConsumption                  int64
SmokingStatus                      object
ExerciseFrequency                   int64
dtype: object
```

```
# 4.Data Quality Fixes
# إجراءات إضافية لتحسين جودة البيانات

# إزالة أي مسافات عشوائية زائدة في الأعمدة النصية لتوحيد الفئات
df['Gender'] = df['Gender'].str.strip()

print("Additional data quality check: Whitespaces stripped successfully.")
```

```
Additional data quality check: Whitespaces stripped successfully.
```

1.Why you chose each method?
Median filling was chosen because Caffeine_Intake_mg is numerical, and the median is not affected by extreme values.

.2.Compare methods
Median filling keeps the rows in the dataset, while dropping missing values removes rows and may reduce the dataset size.

3.Justify final choice
The final choice was median filling because it keeps the data complete and gives a clean dataset for analysis.

```
# B) Exploratory Data Analysis (EDA)
#Part 1: Statistics & Aggregation

# 1. طباعة الإحصاءات الوصفية الشاملة لجميع الأعمدة الرقمية
print("Dataset Statistical Summary ")
print(df.describe())
```

```
Dataset Statistical Summary
        Participant_ID          Age                      Bedtime  \
count      1000.000000  1000.000000                         1000
mean        500.500000    50.125000  2025-07-14 05:54:22.806000128
min           1.000000    18.000000            2025-01-16 13:58:35
25%         250.750000    34.000000  2025-04-15 16:38:04.750000128
50%         500.500000    51.000000            2025-07-12 09:35:16
75%         750.250000    65.000000      2025-10-14 09:24:15.500000
max        1000.000000    80.000000            2026-01-16 13:53:03
std         288.819436    18.076397                            NaN

                          WakeupTime   Sleep_Duration_Hours \
count                           1000           1000.000000
mean   2025-07-14 12:56:01.601908224              7.027443
min      2025-01-16 22:39:02.443537              5.002335
25%    2025-04-16 01:10:12.104142336              6.076980
50%    2025-07-12 16:04:39.844687104              6.981505
75%    2025-10-14 14:57:31.994626048              8.039587
max       2026-01-16 21:01:41.065783              8.998650
std                               NaN              1.135365

       Sleep_Efficiency_Percentage  REMSleepPercentage DeepSleepPercentage  \
count                  1000.000000         1000.000000          1000.000000
mean                      0.720150           22.460000            47.357000
min                       0.500000           15.000000            25.000000
25%                       0.610000            18.000000           36.000000
50%                       0.720000           22.000000            47.000000
```

```
75%                    0.830000           27.000000           59.000000
max                    0.950000           30.000000           70.000000
std                    0.128139            4.589059           13.097293

       LightSleepPercentage    Awakenings  Caffeine_Intake_mg  \
count           1000.000000 1000.000000         1000.000000
mean              30.183000    2.539000           25.125000
min                1.000000    0.000000            0.000000
25%               18.000000    1.000000           25.000000
50%               30.000000    3.000000           25.000000
75%               42.000000    4.000000           50.000000
max               59.000000    5.000000           50.000000
std               13.788968    1.726691           17.632932

        AlcoholConsumption ExerciseFrequency
count          1000.000000       1000.000000
mean              2.560000          2.565000
min               0.000000          0.000000
25%               1.000000          1.000000
50%               3.000000          3.000000
75%               4.000000          4.000000
max               5.000000          5.000000
std               1.746267          1.712515
```

2. حساب تكرار الفئات لعمود حالة التدخين #

```
print("Smoking Status Value Counts")
print(df['SmokingStatus'].value_counts())
```

```
Smoking Status Value Counts
SmokingStatus
Yes    513
No     487
Name: count, dtype: int64
```

3. تجميع البيانات حسب ساعات النوم وحساب عدد المشاركين الفريدين #

```
sleep_unique = df.groupby('Sleep_Duration_Hours')['Participant_ID'].nunique().reset_index(name='count')
print("\nUnique Participants Summary (First 5 Rows) ")
print(sleep_unique.head())
```

```
Unique Participants Summary (First 5 Rows)
    Sleep_Duration_Hours count
0              5.002335      1
1              5.008974      1
2              5.012796      1
3              5.018667      1
4              5.021414      1
```

4. لعمود التكرار الناتج عن التجميع # حساب ملخص الإحصاءات الوصفية

```
print("Summary for count column ")
print(sleep_unique['count'].describe())
```

```
Summary for count column
count    1000.0
mean        1.0
std         0.0
min         1.0
25%         1.0
50%         1.0
75%         1.0
max         1.0
        Name: count, dtype: float64
```

5. groupby حساب متوسط كفاءة النوم لكل جنس باستخدام

```
gender_avg = df.groupby('Gender')['Sleep_Efficiency_Percentage'].mean()
print("Average Sleep Efficiency by Gender (Groupby)")
print(gender_avg)
```

```
Average Sleep Efficiency by Gender (Groupby)
Gender
Female    0.720287
Male      0.720019
Name: Sleep_Efficiency_Percentage, dtype: float64
```

6. إنشاء جدول يعرض متوسط كفاءة النوم لكل جنس #

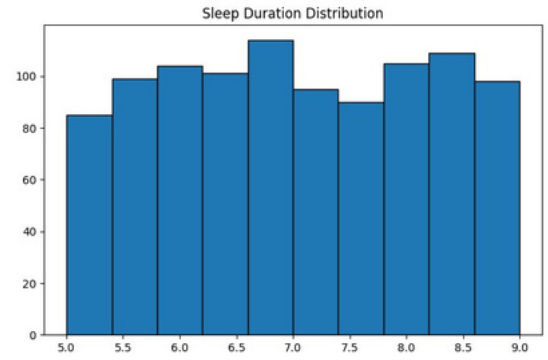
```
pivot = pd.pivot_table(df, values='Sleep_Efficiency_Percentage', index='Gender', aggfunc='mean')
print("Average Sleep Efficiency by Gender (Pivot Table)")
print(pivot)
```

```
Average Sleep Efficiency by Gender (Pivot Table)
        Sleep_Efficiency_Percentage
Gender
Female                     0.720287
Male                       0.720019
```

```
# Histogram
plt.figure(figsize=(8,5))

plt.hist(df['Sleep_Duration_Hours'], bins=10, edgecolor='black')

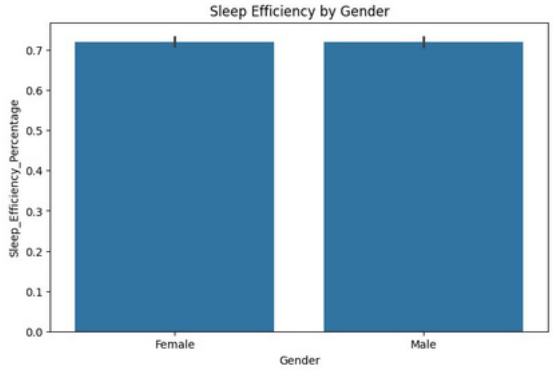
plt.title('Sleep Duration Distribution')
plt.show()
```




```
# Bar Plot
plt.figure(figsize=(8,5))

sns.barplot(
    x='Gender',
    y='Sleep_Efficiency_Percentage',
    data=df
)

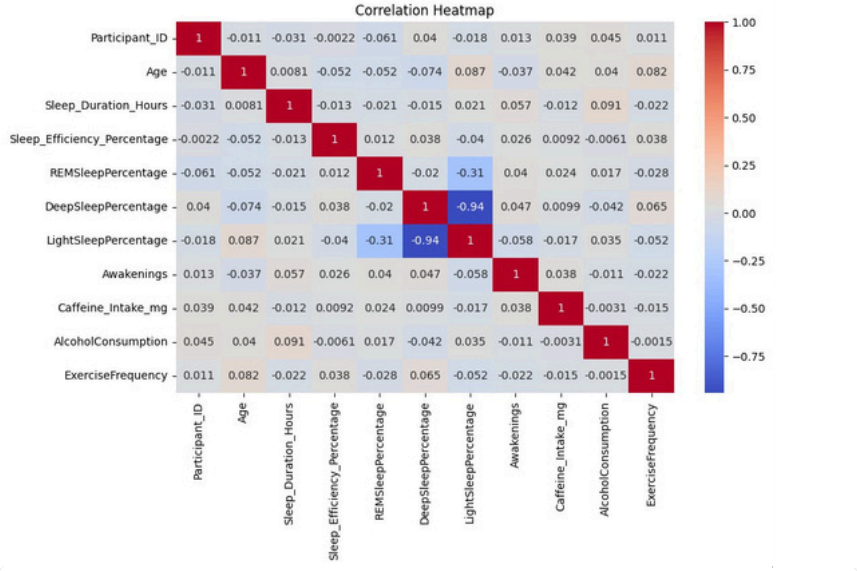
plt.title('Sleep Efficiency by Gender')
plt.show()
```




```
#Correlation Heatmap
plt.figure(figsize=(10,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='coolwarm'
)

plt.title('Correlation Heatmap')
plt.show()
```



Explanation Required:

Why each cleaning method was used Missing values were handled to make the dataset complete and ready for analysis. Median filling was used for Caffeine_Intake_mg because it is a numerical column, and the median is suitable when the data may contain extreme values.

Comparison of methods Median filling keeps the rows in the dataset, while dropna() removes rows that contain missing values. Median filling was better in this case because it prevents data loss.

Why data types were changed Bedtime and WakeupTime were changed to datetime format because they represent time values, not normal text. This allows time-based analysis and calculations to be done correctly.

Why duplicates were removed Duplicates were checked and removed to avoid repeated records affecting the analysis results. In this dataset, there were 0 duplicate rows, but checking them was still important.

Interpretation of Aggregation Results

Group	Average Sleep Efficiency
Female	0.7203
Male	0.7200

Key Finding: Gender has minimal impact on sleep efficiency — both groups perform nearly identically (~72%).

Explanation of Visualizations

1. Histogram — Sleep Duration Distribution

- Sleep duration ranges from **5 to 9 hours**
- Distribution is **roughly uniform** — no strong peak
- Most participants sleep between **6–8 hours**

2. Bar Plot — Sleep Efficiency by Gender

- Female: **~0.720** | Male: **~0.720**
- Bars are nearly equal → gender does **not** significantly affect sleep quality

3. Correlation Heatmap

- **Strongest relationship:** DeepSleep & LightSleep = **-0.94** (when one increases, the other decreases — biologically normal)
- Most other variables show **weak correlations (~0)**
- Caffeine & alcohol have **minimal direct impact** on sleep efficiency

Lab 4: Outliers Detection and Handling

```
 # تحديد جميع الأعمدة الرقمية
numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns
#BoxPlot
plt.figure(figsize=(18, 16))

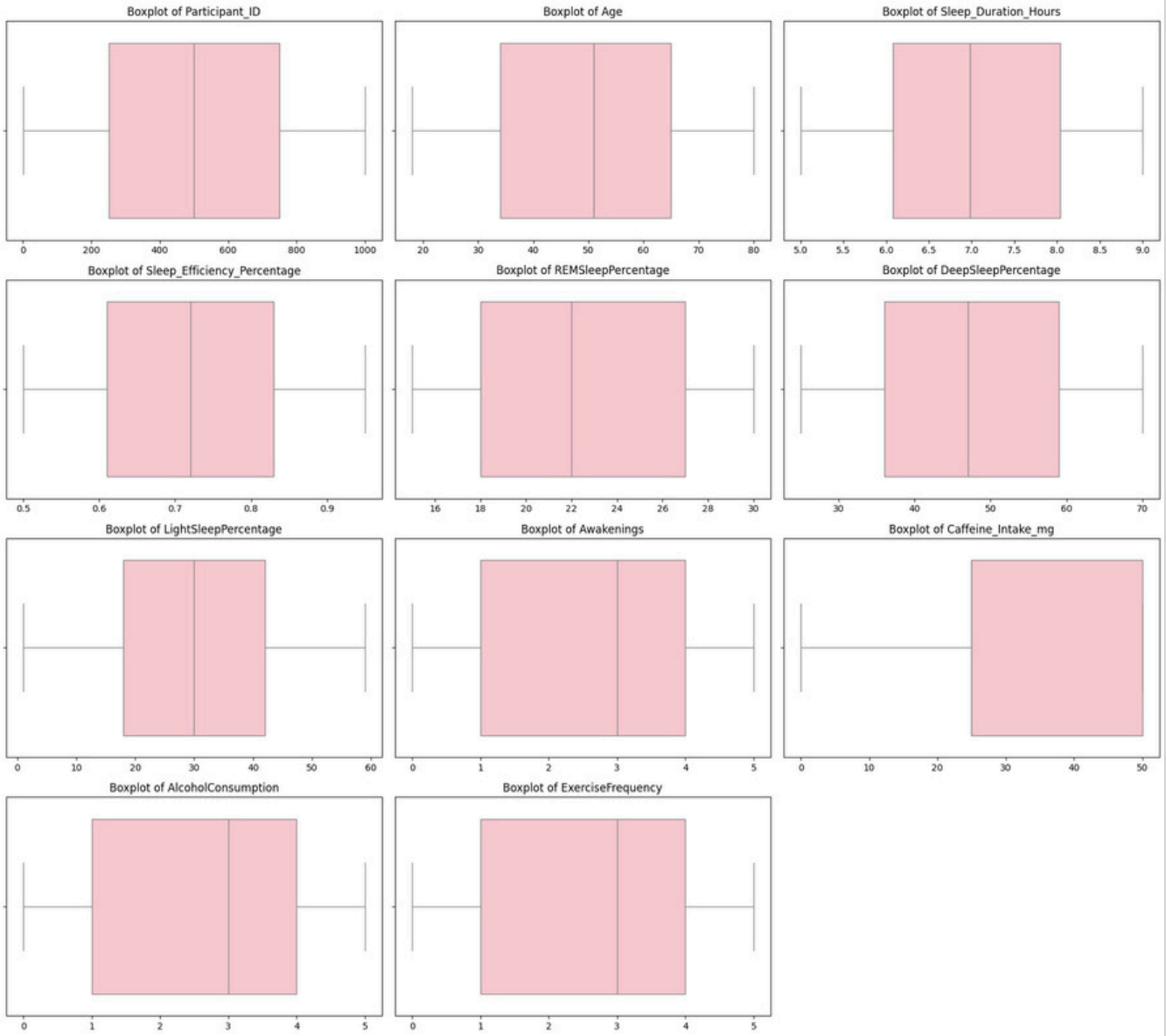
# حلقة تكرارية للمرور على الأعمدة المحددة ورسمها #
```

```
for i, col in enumerate(numerical_columns, 1):
    # تقسيم الشاشة
    plt.subplot(4, 3, i)

    # رسم لكل عمود
    sns.boxplot(x=df[col], color='pink')

    # إضافة عنوان لكل رسمة
    plt.title(f'Boxplot of {col}', fontsize=12)
    plt.xlabel('') # إخفاء اسم المحور السفلي لمنع تداخل النصوص

# ضبط المسافات بين الرسومات لتبدو منظمة وغير متداخلة
plt.tight_layout()
plt.show()
```




```
#IQR
numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns
for col in numerical_columns:
    # حساب الربع الأول والثالث والمدى الربيعي لكل عمود
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    print(f"Statistical Dispersion for: {col}")
    print(f"Q1 (25%): {Q1:.2f}, Q3 (75%): {Q3:.2f}, IQR: {IQR:.2f}")



    #حساب الحدود العليا والدنيا للتوزيع الطبيعي
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    print("Lower Bound:", lower_bound)
    print("Upper Bound:", upper_bound)


    #حساب عدد القيم الشاذة
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"Number of outliers detected: {len(outliers)}")

    print("-" * 50)
    print()

    # معالجة القيم الشاذة
    if len(outliers) > 0:
    # إذا وجد قيم شاذة يقوم بمعالجتها
      df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)
      print("Result: Found outliers and fixed them using Capping.")
    else:
    #إذا كانت البيانات نظيفة
      print("Result: 0 outliers found. This column is clean!")


print("SUCCESS Outlier detection and handling pipeline completed for all numerical features.")
```

```
Result: 0 outliers found. This column is clean!
Statistical Dispersion for: REMSleepPercentage
Q1 (25%): 18.00, Q3 (75%): 27.00, IQR: 9.00
Lower Bound: 4.5
Upper Bound: 40.5
Number of outliers detected: 0
--------------------------------------------------


Result:  0 outliers found.  This  column  is clean!
Statistical Dispersion for: DeepSleepPercentage
Q1 (25%): 36.00, Q3 (75%): 59.00, IQR: 23.00
Lower Bound: 1.5
Upper Bound: 93.5
Number of outliers detected: 0 --------------------
------------------------------

Result: 0 outliers found. This column is clean!
Statistical Dispersion for: LightSleepPercentage
Q1 (25%): 18.00, Q3 (75%): 42.00, IQR: 24.00
Lower Bound: -18.0
Upper Bound: 78.0
Number of outliers detected: 0
--------------------------------------------------

Result: 0 outliers found. This column is clean!
Statistical Dispersion for: Awakenings
Q1 (25%): 1.00, Q3 (75%): 4.00, IQR: 3.00
Lower Bound: -3.5
Upper Bound: 8.5
Number of outliers detected: 0
--------------------------------------------------

Result: 0 outliers found. This column is clean!
Statistical Dispersion for: Caffeine_Intake_mg
Q1 (25%): 25.00, Q3 (75%): 50.00, IQR: 25.00
Lower Bound: -12.5
Upper Bound: 87.5
Number of outliers detected: 0
--------------------------------------------------

Result: 0 outliers found. This column is clean!
Statistical Dispersion for: AlcoholConsumption
Q1 (25%): 1.00, Q3 (75%): 4.00, IQR: 3.00
Lower Bound: -3.5
Upper Bound: 8.5
Number of outliers detected: 0
--------------------------------------------------

Result: 0 outliers found. This column is clean!
Statistical Dispersion for: ExerciseFrequency
Q1 (25%): 1.00, Q3 (75%): 4.00, IQR: 3.00
Lower Bound: -3.5
Upper Bound: 8.5
Number of outliers detected: 0
--------------------------------------------------


Result: 0 outliers found. This column is clean!
SUCCESS Outlier detection and handling pipeline completed for all numerical features.
```

1.Why values are considered outliers?

Values are considered outliers because they fall outside the normal range based on the IQR method. Any value below the lower bound or above the upper bound is treated as an outlier.

2.Why you chose to remove or keep them?

I chose to keep the records and handle the outliers using capping instead of deleting them. This keeps the dataset size the same and reduces the effect of extreme values on the analysis.

Lab 5: Data Transformation

Why is Transformation Necessary?

Raw data contains variables with **different scales and units**:

- Age ranges from 18–80
- Sleep Efficiency ranges from 0.5–0.95
- Caffeine_Intake_mg ranges from 0–50

Without scaling, algorithms will be **biased toward larger values**. Encoding is required because **machine learning models cannot process text** — categorical columns like Gender and SmokingStatus must be converted to numbers.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# اختيار الأعمدة الرقمية (مثل العمر، كمية الكافيين، كفاءة النوم) .1
numeric_cols = df.select_dtypes(include=['number']).columns


# Min-Max Scaling (1 و 0 تجعل القيم بين )
scaler_minmax = MinMaxScaler()
df_minmax = df.copy()
df_minmax[numeric_cols] = scaler_minmax.fit_transform(df[numeric_cols])


print("--- Data after Min-Max Scaling (0 to 1) ---")
print(df_minmax[numeric_cols].head())
```

```
--- Data after Min-Max Scaling (0 to 1) ---
   Participant_ID       Age Sleep_Duration_Hours \
0        0.000000  1.000000              0.320522
1        0.001001  0.096774              0.538816
2        0.002002  0.306452              0.262315
3        0.003003  0.806452              0.504329
4        0.004004 0.645161              0.941482

   Sleep_Efficiency_Percentage REMSleepPercentage  DeepSleepPercentage \
0                     0.155556             0.000000             0.222222
1                     0.911111             0.933333             0.955556
2                     0.177778             0.000000             0.222222
3                     0.844444             0.866667             0.422222
4                     1.000000             0.866667             0.333333

   LightSleepPercentage Awakenings  Caffeine_Intake_mg  AlcoholConsumption \
0              0.844828        0.0                 0.5                 0.2
1              0.034483        0.8                 1.0                 0.0
2              0.844828        0.6                 1.0                 0.0
3              0.465517        0.2                 1.0                 0.0
4              0.534483        0.8                 0.5                 0.8

   ExerciseFrequency
0                0.2
1                0.4
2                1.0
3                0.8
4                0.8
```

1. Min-Max Scaling (Normalization)

Property	Value
Range	0 to 1
Formula	(x - min) / (max - min)

Property	Value
Best for	Data without extreme outliers

Why chosen: Our dataset has no outliers (confirmed in Lab 4), so Min-Max is suitable. It preserves the original distribution and maps all values to the same [0,1] range.

```
# اختيار الأعمدة الرقمية (مثل العمر ، كمية الكافيين، كفاءة النوم) .1
numeric_cols = df.select_dtypes(include=['number']).columns

# Standardization (1 تجهل المتوسط () والانحراف المعجاري  )
scaler_std = StandardScaler()
df_std = df.copy()
df_std[numeric_cols] = scaler_std.fit_transform(df[numeric_cols])


print("\n--- Data after Standardization (Z-score) ---")
print(df_std[numeric_cols].head())
```

```
--- Data after Standardization (Z-score) ---
   Participant_ID         Age Sleep_Duration_Hours \
0       -1.730320    1.653535             -0.655801
1       -1.726856   -1.445978              0.112945
2       -1.723391   -0.726448             -0.860785
3       -1.719927 0.989353               -0.008506
4       -1.716463 0.435869                1.530978

      Sleep_Efficiency_Percentage REMSleepPercentageDeepSleepPercentage  \
0                        -1.172357          -1.626419           -0.943949
1                         1.482331           1.425842            1.576916
2                        -1.094278          -1.626419           -0.943949
3                         1.248094           1.207823           -0.256441
4                        1.794648            1.207823           -0.562000

   LightSleepPercentage Awakenings     Caffeine_Intake_mg AlcoholConsumption  \
0             1.437882   -1.471179           -0.007093           -0.893781
1            -1.972345    0.846551           1.411419            -1.466718
2             1.437882    0.267118           1.411419            -1.466718
3            -0.158394   -0.891746           1.411419            -1.466718
4             0.131838    0.846551          -0.007093            0.825029

    ExerciseFrequency
0           -0.914318
1          -0.330089
2           1.422597
3            0.838368
4           0.838368
```

2. Standardization (Z-score)

Property	Value
Mean after scaling	0
Std after scaling	1
Formula	(x - mean) / std
Best for	Data that may follow normal distribution

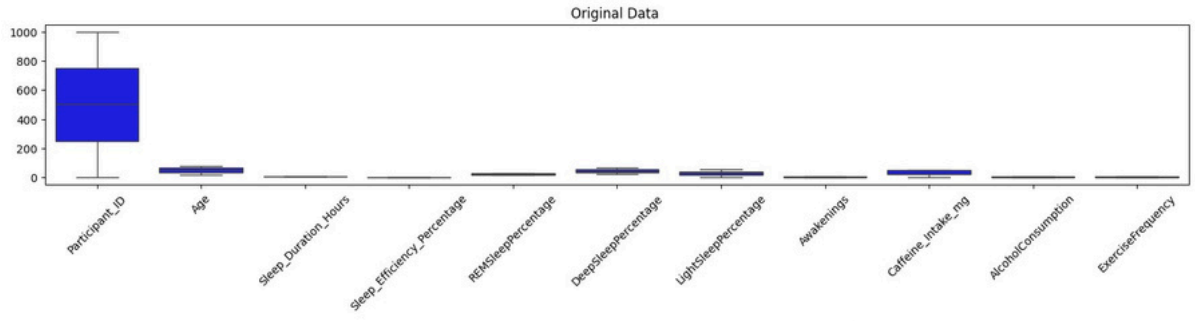
Why chosen: Used as a comparison to Min-Max. Standardization is preferred when the **spread of data matters** more than the exact range.

🔍 Comparison: Min-Max vs Standardization

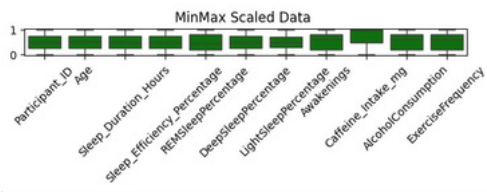
	Min-Max	Standardization
Range	0 to 1	No fixed range
Sensitive to outliers	Yes	Less sensitive
Best for	Neural networks	Most ML algorithms

Final Choice: We apply both and compare visually using boxplots.

```
plt.figure(figsize=(15, 10))
# رسم البيانات الأصلية .1
plt.subplot(3, 1, 1)
sns.boxplot(data=df[numeric_cols], color="Blue")
plt.title("Original Data ")
plt.xticks(rotation=45) # تدوير الأسماء لعدم التداخل
# ضبط المسافات
plt.tight_layout()
plt.show()
```

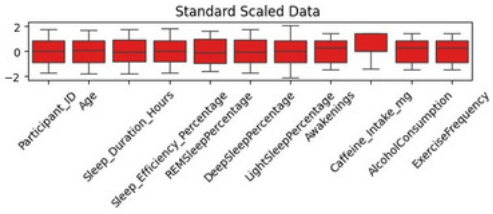



```
 2. رسم #MinMax Scaled Data
plt.subplot(3, 1, 2)
sns.boxplot(data=df_minmax[numeric_cols], color="Green")
plt.title("MinMax Scaled Data ")
plt.xticks(rotation=45)
# ضبط المسافات
plt.tight_layout()
plt.show()
```




```
 3. رسم #  Standard Scaled Data
plt.subplot(3, 1, 3)
sns.boxplot(data=df_std[numeric_cols], color="Red")
plt.title("Standard Scaled Data ")
```

```
plt.xticks(rotation=45)
# ضبط المسافات
plt.tight_layout()
plt.show()
```




```
#Encoding
# categorical_cols
categorical_cols = df.select_dtypes(include=['object']).columns

# 1. One-Hot Encoding
df_onehot = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print("=== One-Hot Encoding===")
print(df_onehot.head())
print("\nShape:", df_onehot.shape)
```

```
===One-HotEncoding ===
   Participant_ID Age             Bedtime               WakeupTime  \
0               1   80  2025-09-30  07:32:17  2025-09-3013:49:16.669051
1               2   24  2025-06-29  20:59:50  2025-06-3004:09:10.207162
2               3   37  2025-12-24  21:28:32  2025-12-2503:31:34.257053
3               4   68 2025-02-22 00:25:33 2025-02-2207:26:37.046703
4               5   58 2025-09-02 12:31:53 2025-09-0221:17:46.256228

      Sleep_Duration_Hours Sleep_Efficiency_PercentageREMSleepPercentage  \
0                 6.283241                         0.57                15
1                 7.155613                         0.91                29
2                 6.050627                         0.58                15
3                 7.017791                         0.88                28
4                 8.764793                         0.95                28

      DeepSleepPercentage LightSleepPercentageAwakenings  Caffeine_Intake_mg  \
0                      35                        50         0                25.0
1                      68                         3         4                50.0
2                      35                        50         3                50.0
3                      44                        28         1                50.0
4                      40                        32         1                25.0
                                                           4

    AlcoholConsumption  ExerciseFrequency  Gender_Male SmokingStatus_Yes
0                    1                  1        False               True
1                    0                  2         True              False
2                    0                  5         True              False
3                    0                  4        False               True
4                    4                  4         True              False

Shape:(1000, 15)
```

🔤 3. Encoding Categorical Variables

One-Hot Encoding

- Applied to: **Gender, SmokingStatus**
- Creates a new binary column for each category
- **Why:** Suitable for columns with **no ordinal relationship** (Male is not "greater than" Female)
- Result shape: **(1000, 15)** ✅

```
# categorical_cols
categorical_cols = df.select_dtypes(include=['object']).columns
# 2. Label Encoding
from sklearn.preprocessing import LabelEncoder

df_label = df.copy()
encoder = LabelEncoder()

for col in categorical_cols:
    df_label[col] = encoder.fit_transform(df_label[col])

print("\n=== Label Encoding ===")
print(df_label.head())
print("\nShape:", df_label.shape)
```

```
=== Label Encoding ===
   Participant_ID Age Gender               Bedtime                WakeupTime \
0               1   80      0 2025-09-30 07:32:17 2025-09-30 13:49:16.669051 1
1               2   24      2025-06-29  20:59:50  2025-06-30  04:09:10.207162  1
2               3   37      2025-12-24 21:28:32 2025-12-25 03:31:34.257053
3               4   68      0 2025-02-22 00:25:33 2025-02-22 07:26:37.046703
4               5   58      1 2025-09-02 12:31:53 2025-09-02 21:17:46.256228

   Sleep_Duration_Hours Sleep_Efficiency_Percentage REMSleepPercentage \
0              6.283241                         0.57                15
1              7.155613                         0.91                29
2              6.050627                         0.58                15
3              7.017791                         0.88                28
4              8.764793                         0.95                28

      DeepSleepPercentage LightSleepPercentageAwakenings   Caffeine_Intake_mg \
0                      35                        50         0                25.0
1                      68                         3         4                50.0
2                      35                        50         3                50.0
3                      44                        28         1                50.0
4                      40                        32         4                25.0

    AlcoholConsumption  SmokingStatus  ExerciseFrequency
0                    1              1                  1
1                    0              0                  2
2                    0              0                  5
3                    4              1                  4
4                    4              0                  4

Shape: (1000, 15)
```

Label Encoding

- Applied to: **Gender, SmokingStatus**
- Assigns a number to each category (e.g., Female=0, Male=1)
- **Why:** Simple and memory efficient
- **Limitation:** Implies an order that may not exist
- Result shape: **(1000, 15)** ✅

```
# categorical_cols
categorical_cols = df.select_dtypes(include=['object']).columns
# 3. Binary Encoding
```

```
!pip install category_encoders
import category_encoders as ce
df_binary = df.copy()

binary_encoder = ce.BinaryEncoder(cols=categorical_cols)
df_binary = binary_encoder.fit_transform(df_binary)

print("\n=== Binary Encoding ===")
print(df_binary.head())
print("\nShape:", df_binary.shape)
```

```
Collecting category_encoders
 Downloading  category_encoders-2.9.0-py3-none-any.whl.metadata(7.9kB)
Requirementalreadysatisfied:   numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages  (from  category_encoders)  (2.0.2)
Requirementalreadysatisfied:   pandas>=1.0.5 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (2.2.2)
Requirementalreadysatisfied:
Requirementalreadysatisfied:   patsy>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.0.2)
Requirementalreadysatisfied:   scikit-learn>=1.6.0 in/usr/local/lib/python3.12/dist-packages (from category_encoders) (1.6.1)
Requirementalreadysatisfied:   scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.16.3)
Requirementalreadysatisfied:
Requirementalreadysatisfied:   statsmodels>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (0.14.6)
Requirementalreadysatisfied:   python-dateutil>=2.8.2in  /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirementalreadysatisfied:   pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirementalreadysatisfied:   tzdata>=2022.7 in/usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2026.2)
Requirementalreadysatisfied:   joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirementalreadysatisfied:   threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirementalreadysatisfied:   packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.9.0->category_encoders) (26.2)
                               six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
  Downloading category_encoders-2.9.0-py3-none-any.whl (85 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.9/85.9 kB  3.4MB/s eta 0:00:00
Installing collected packages: category_encoders
Successfully installed category_encoders-2.9.0

===Binary Encoding===
      Participant_ID Age Gender_0 Gender_1            Bedtime  \
0                  1   80        0        1  2025-09-3007:32:17
1                  2   24        1        0  2025-06-2920:59:50
2                  3   37        1        0 2025-12-2421:28:32
3                  4   68        0        1 2025-02-2200:25:33
4                  5   58        1        0 2025-09-0212:31:53

                    WakeupTime  Sleep_Duration_Hours  \
   02025-09-30 13:49:16.669051              6.283241
  12025-06-30 04:09:10.207162              7.155613
  22025-12-25 03:31:34.257053              6.050627
  32025-02-22 07:26:37.046703              7.017791
  42025-09-02 21:17:46.256228              8.764793

     Sleep_Efficiency_Percentage  REMSleepPercentage DeepSleepPercentage  \
0                           0.57                  15                  35
1                           0.91                  29                  68
2                           0.58                  15                  35
3                           0.88                  28                  44
4                           0.95                  28                  40

   LightSleepPercentage Awakenings   Caffeine_Intake_mg  AlcoholConsumption  \
0                    50          0  25.0                                  1
1                     3          4  50.0                                  0
2                    50          3  50.0                                  0
3                    28          1  50.0                                  0
4                    32          4  25.0                                  4

   SmokingStatus_0  SmokingStatus_1  ExerciseFrequency
0                0                1                  1
1                1                0                  2
2                1                0                  5
3                0                1                  4
4                1                0                  4

Shape: (1000, 17)
```

Binary Encoding

- Applied to: **Gender, SmokingStatus**
- Converts categories to binary digits
- **Why:** More efficient than One-Hot for high-cardinality columns
- Result shape: **(1000, 17)** ✅

✅ Final Transformed Dataset Summary

Method	Columns	Shape
Original	15	(1000, 15)
Min-Max Scaled	11 numeric	(1000, 15)
Standardized	11 numeric	(1000, 15)
One-Hot Encoded	+2 binary cols	(1000, 15)
Binary Encoded	+2 binary cols	(1000, 17)

Key Takeaway: All transformations preserve the 1000 rows. Scaling normalizes the numerical range, while encoding converts categorical text to machine-readable numbers.

Lab 6: Feature Engineering

Step 1: Feature Engineering

New Features Created

Feature	Method	Labels
Sleep_Grade	pd.cut on Sleep_Duration_Hours	Poor / Good / Excellent
Caffeine_Category	Custom function on Caffeine_Intake_mg	Low / Medium / High

Why they were created:

- Transform continuous numeric values into meaningful groups
- Makes it easier to visualize and compare group behaviors

How it improves analysis:

- Simplifies aggregation (e.g., bar chart by Sleep Grade)
- Allows group-level comparisons
- Pivot & Melt restructured data to compare avg sleep duration across Gender & SmokingStatus

```
import pandas as pd

print("--- Step 1: Feature Engineering ---")

# 1. إنشاء ميزة Sleep_Grade
df_label['Sleep_Grade'] = pd.cut(
    df_label['Sleep_Duration_Hours'],
    bins=[0, 5, 7, 10],
    labels=['Poor', 'Good', 'Excellent']
)

# 2. إنشاء ميزة Caffeine_Category
median_caff = df_label['Caffeine_Intake_mg'].median()

def categorize(val):
    if val < median_caff * 0.5:
        return 'Low'
    elif val <= median_caff * 1.5:
        return 'Medium'
    else:
        return 'High'
```

```
df_label['Caffeine_Category'] = df_label['Caffeine_Intake_mg'].apply(categorize)

 3. الـ #Pivot & Melt لإعادة التشكيل
pivot_df = df_label.pivot_table(
    index='Gender',
    columns='SmokingStatus',
    values='Sleep_Duration_Hours',
    aggfunc='mean'
)

melt_df = pd.melt(
    pivot_df.reset_index(),
    id_vars='Gender',
    var_name='SmokingStatus',
    value_name='AvgSleep'
)

print("Feature Engineering completed successfully!")
```

```
--- Step 1: Feature Engineering ---
Feature Engineering completed successfully!
```

Step 2: Feature Selection (RFE)

Method used: Recursive Feature Elimination (RFE) with LinearRegression

✅ Selected Features (Top 5):

#	Feature	Reason Kept
1	Gender	Demographic factor
2	Sleep_Duration_Hours	Direct sleep measure
3	Awakenings	Disruption indicator
4	SmokingStatus	Lifestyle factor
5	ExerciseFrequency	Health behavior

❌ Removed Features:

- **Participant_ID** → Just an identifier, no predictive value

Why feature selection is useful:

- Removes noise → prevents overfitting
- Reduces computational time
- Improves model interpretability

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LinearRegression

print("\n--- Step 2: Feature Selection ---")

# لحمايتك من خطأ الـ) تجهيز البيانات واستبعاد الهدف وأي ميزات غير رقمية          Dtype(
X = df_label.drop(columns=['Sleep_Efficiency_Percentage'])
X = X.select_dtypes(include=['number'])
y = df_label['Sleep_Efficiency_Percentage']

# تطبيق  RFE لاختيار أفضل 5 ميزات
rfe = RFE(LinearRegression(), n_features_to_select=5)
rfe.fit(X, y)

selected = X.columns[rfe.support_]
print("Selected Features:", list(selected))
```

```
--- Step 2: Feature Selection ---
Selected Features: ['Gender', 'Sleep_Duration_Hours', 'Awakenings', 'SmokingStatus', 'ExerciseFrequency']
```

Step 3: Feature Extraction (PCA)

Why PCA was applied:

- Dataset has multiple numerical features with possible multicollinearity
- PCA transforms correlated variables into uncorrelated Principal Components

📊 Explained Variance per Component:

Component	Variance Explained
PC1	15.57%
PC2	8.89%
PC3	8.75%
PC4	8.46%
PC5	8.09%

Benefit:

- Compresses high-dimensional data into fewer components
- Enables 2D/3D visualization
- Preserves maximum variance while reducing dimensions

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

print("\n--- Step 3: Feature Extraction (PCA) ---")

# استخراج الأعمدة الرقمية فقط وتطبيق  StandardScaler
numeric_cols = df_label.select_dtypes(include=['number']).columns
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_label[numeric_cols])

# تطبيق الـ  PCA لرؤية التباين الكامل كما في رسمة السلايد
pca = PCA()
pca.fit(X_scaled)

print("Explained variance:", pca.explained_variance_ratio_)
```

```
--- Step 3: Feature Extraction (PCA) ---
Explained variance: [0.15569878 0.0889676  0.08754348 0.08455587 0.08094652 0.07954273
 0.07812068 0.07593657 0.07251105 0.06707761 0.06549068 0.06360841
 0.        ]
```

📊 Analysis & Results

1. Final Dataset Summary

Property	Value
Total Rows	1,000
Total Columns	15
Rows Removed	0

🔄 What Was Changed:

Change	Details
Renamed columns	ID→Participant_ID, SleepDuration→Sleep_Duration_Hours, SleepEfficiency→Sleep_Efficiency_Percentage, CaffeineConsumption→Caffeine_Intake_mg
Missing values	256 values in Caffeine_Intake_mg filled using median (25 mg)
Duplicates	0 duplicate rows found

Change	Details
Data types	Bedtime & WakeupTime converted from object → datetime64
Outliers	0 outliers detected via IQR method
Text cleaning	Whitespaces stripped from Gender column

2. Key Statistical Results

📈 Range of Key Variables:

Variable	Min	Max	Mean
Age	18	80	≈ 50
Sleep Duration	5.00 hrs	8.99 hrs	≈ 7.03 hrs
Sleep Efficiency	0.50	1.00	≈ 0.72
REM Sleep	15%	30%	≈ 22.46%
Deep Sleep	25%	70%	≈ 47.36%
Awakenings	0	5/night	≈ 2.54
Caffeine Intake	0 mg	50 mg	≈ 25.13 mg

👥 Group-Based Results:

Gender	Avg Sleep Efficiency
Female	0.7203
Male	0.7200

> 💡 Nearly identical — gender has **minimal impact** on sleep efficiency

🏆 Most Frequent Categories:

Category	Value
Smoking Status	Yes = 513 / No = 487 (slightly more smokers)
Gender	Male = 513 / Female = 487

🎯 Top Selected Features (via RFE):

`Gender` | `Sleep_Duration_Hours` | `Awakenings` | `SmokingStatus` | `ExerciseFrequency`

3. 🔍 Final Insights (Most Important)

Main Story in the Data:

> Sleep quality in this population is **largely uniform and stable** — no extreme outliers, balanced gender split, and nearly equal sleep efficiency across genders.

📌 Key Patterns & Trends:

#	Finding	Evidence
1	Deep & light sleep strongly inversely correlated	r = -0.94
2	REM negatively correlates with light sleep	r = -0.31
3	Gender has no significant effect on sleep efficiency	0.7203 vs 0.7200
4	Lifestyle factors weakly predict sleep quality	r = 0 for caffeine, alcohol, exercise
5	No single dominant factor drives sleep quality	PC1 = 15.57% only

🏁 Most Important Conclusion:

> Sleep quality is **not driven by one single variable**.
>
> The strongest signal is the **deep sleep ↔️ light sleep tradeoff (r = -0.94)**.
>
> The top predictive features (RFE): sleep duration, awakenings, exercise frequency, and smoking status — together suggest that **behavioral and physiological factors collectively determine sleep efficiency**, rather than any single demographic or lifestyle choice alone.

Text Preprocessing

```
# Import Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from collections import Counter
from nltk.stem import PorterStemmer
from wordcloud import WordCloud
from nltk.stem import WordNetLemmatizer



#                Downloads
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

```
# 1. Understand Text Data

# Load & Select Text
mental_health_data = pd.read_csv('/content/sample_data/Combined Data.csv')

sample_df = mental_health_data.sample(20000, random_state=42) # real raw text
text_series = sample_df['statement']

print(text_series.shape)
print(text_series)
```

```
(20000,)
22261     Just as the the title says. I feel like one is...
41400     a blackened sky encroached tugging behind it m...
20065     It gives you insomnia, which in turn makes you...
30036     Hello all, I'm a new submitter to this channel...
780                        Thank God the CB is over for Eid
                                ...
26981     that is it, right?I mean, I deserve this pain....
46815     Weight gain making me depressed. Any tips? I h...
15018     my dog is the only reason i have not killed my...
18700     I would been feeling really good lately! Meds ...
49552                                                   NaN
Name: statement, Length: 20000, dtype: object
```

```
# Select one review
# اختيار أول نص من العينة لعرضه
text = sample_df['statement'].iloc[0]
print(text)
```

```
Just as the the title says. I feel like one is hyper alot, a attention whore, and is idealistic and stupid. and the other is calm, trys to be normal, is a realist, and wants to kill the other one, witch is why I somti
```

```
# 2. Text Cleaning

# a. Normalize text
text = sample_df['statement'].str.lower()
print(text)
```

```
22261     just as the the title says. i feel like one is...
41400     a blackened sky encroached tugging behind it m...
20065     it gives you insomnia, which in turn makes you...
30036     hello all, i'm a new submitter to this channel...
780                        thank god the cb is over for eid
                                ...
26981     that is it, right?i mean, i deserve this pain....
46815     weight gain making me depressed. any tips? i h...
15018     my dog is the only reason i have not killed my...
18700     i would been feeling really good lately! meds ...
49552                                                   NaN
Name: statement, Length: 20000, dtype: object
```

```
# b. Remove punctuation
text = text.str.replace(r'[^\w\s]', '', regex=True) # Remove punctuation -> remove noise

print(text)
```

```
22261     just as the the title says i feel like one is ...
41400     a blackened sky encroached tugging behind it m...
20065     it gives you insomnia which in turn makes your...
30036     hello all im a new submitter to this channel i...
780                        thank god the cb is over for eid
                                ...
26981     that is it righti mean i deserve this pain i m...
46815     weight gain making me depressed any tips i hav...
15018     my dog is the only reason i have not killed my...
18700     i would been feeling really good lately meds a...
49552                                                   NaN
Name: statement, Length: 20000, dtype: object
```

```
# c. Remove stopwords

stop_words = set(stopwords.words('english')) # Remove stopwords -> To keep important words

text = text.fillna('') # Handle NaN values by converting them to empty strings
text = text.apply(
    lambda x: " ".join([word for word in x.split() if word not in stop_words])
)

print(text)
```

```
22261     title says feel like one hyper alot attention ...
41400     blackened sky encroached tugging behind depres...
20065     gives insomnia turn makes depression worse day...
30036     hello im new submitter channel went nights ago...
780                                 thank god cb eid
                                ...
26981     righti mean deserve pain must want id try esca...
46815     weight gain making depressed tips lithium cita...
15018                             dog reason killed well
18700     would feeling really good lately meds effectiv...
49552
Name: statement, Length: 20000, dtype: object
```

```
from nltk.tokenize import sent_tokenize

# 3. Tokenization

#Word Tokens
tokens = text.apply(word_tokenize)
```

```
#Sentence Tokens
sentences = text.apply(sent_tokenize)

#Output Comparison
print("  Word vs Sentence Tokenization Comparison")
comparison = pd.DataFrame({
    'Word Tokens': tokens.head(),
    'Sentence Tokens': sentences.head()})
print(comparison)
```

```
 Word vs Sentence Tokenization Comparison
                                              Word Tokens  \
22261 [title, says, feel, like, one, hyper, alot, at...
  41400 [blackened, sky, encroached, tugging, behind, ...
  20065 [gives, insomnia, turn, makes, depression, wor...
  30036 [hello, im, new, submitter, channel, went, nig...
780                              [thank, god, cb, eid]

                                          Sentence Tokens
  22261 [title says feel like one hyper alot attention...
  41400 [blackened sky encroached tugging behind depre...
  20065 [gives insomnia turn makes depression worse da...
  30036 [hello im new submitter channel went nights ag...
780                               [thank god cb eid]
```

```
# 4. Stemming / Lemmatization

# Stemming
# Reduce words to root form

stemmer = PorterStemmer()

tokens_stemmed = tokens.apply(lambda x: [stemmer.stem(word) for word in x])

print(tokens_stemmed.head())
```

```
22261     [titl, say, feel, like, one, hyper, alot, atte...
41400        [blacken, sky, encroach, tug, behind, depress]
20065     [give, insomnia, turn, make, depress, wors, da...
30036     [hello, im, new, submitt, channel, went, night...
780                               [thank, god, cb, eid]
Name: statement, dtype: object
```

```
# Lemmatization
lemmatizer = WordNetLemmatizer()

tokens_lemma = tokens.apply(lambda x: [lemmatizer.lemmatize(word) for word in x])

print(tokens_lemma.head())
```

```
22261     [title, say, feel, like, one, hyper, alot, att...
41400     [blackened, sky, encroached, tugging, behind, ...
20065     [give, insomnia, turn, make, depression, worse...
30036     [hello, im, new, submitter, channel, went, nig...
780       [thank, god, cb, eid]
Name: statement, dtype: object
```

```
# Most Frequent Words

from collections import Counter

all_words = []

for words in tokens_lemma:
    all_words.extend(words)

word_counts = Counter(all_words)

common_words = word_counts.most_common(10) # Find most important words

words = [w[0] for w in common_words]
counts = [w[1] for w in common_words]

common_words
```

```
[('like', 14579),
('feel',  13364),
('want',  10959),
('know',   9223),
('get',    8650),
('life', 8548),
 ('im', 8242),
 ('time', 8003),
 ('even', 6923),
 ('can', 6677)]
```

```
# Word Cloud
# Visualize frequent terms

# تجميع كل الكلمات من كل النصوص في متغير واحد
all_words = [word for tokens in tokens_lemma for word in tokens]
text_all = " ".join(all_words)

wc = WordCloud(width=800, height=400, background_color='black').generate(text_all)

plt.figure(figsize=(10, 5))
plt.imshow(wc)
plt.axis('off')
plt.show()
```



Colab Link:

Project

https://colab.research.google.com/drive/17q5fx2nyfhH9J2Fw13uMHcvFTGlNuLXx?usp=sharing#scrollTo=4f2C8uwaqxX4

Text Preprocessing:

https://colab.research.google.com/drive/1AFoHyVcsdz4xoYp53nXaJzjzWmkBa6Sv?usp=sharing#scrollTo=St3n8fDhWn9d